Oct 21, 2026 · @E

What you're building

Build a complete, cross-platform music player for a large library of local files. It needs hi-res audio, first-class metadata editing and an artwork-forward design inspired by Apple Music. Every device I own keeps its own copy of the library, and a change made on any of them reaches all the others.

This is a large project: plan it, build it in milestones, and keep every milestone working and tested. Where this spec is ambiguous, choose what a demanding listener with a 200,000-track library would want, and record the decision.

Project settings

These settings drive the decisions below; if anything else in this spec conflicts with them, the settings win.

- **Working name:** Cadenza (placeholder; rename freely).
- **Desktop:** Windows, macOS and Linux.
- **Mobile:** iOS and Android.
- **Cloud storage:** my own cloud account (Dropbox, Google Drive, OneDrive or S3-compatible storage such as Backblaze B2) is the sync hub and off-site backup; no always-on server is needed.
- **Library size:** design for up to 200,000 tracks, mixed lossy and lossless, including 24-bit/192 kHz PCM and DSD.
- **Where the files live:** on every device. Each one keeps its own full copy, and a change made on any device reaches all the others.
- **Priorities, in order:** never damage my files, then sound quality, metadata quality, design polish and breadth of features.

How to work

Plan first, then build in milestones that each leave a working, tested app.

1. Before writing code, create `docs/ARCHITECTURE.md` and `docs/ROADMAP.md`. Architecture covers components, the stack with current versions and licences, data model, audio pipeline, sync design and per-platform limits. The roadmap breaks the

milestones below into tasks. Then stop and give me a short summary to approve.
2. Build one milestone at a time, as vertical slices. Each must build, run and pass its tests on every platform it touches before you move on. Commit at each working step.
3. Keep `docs/PROGRESS.md` current: done, next, known issues and decisions. A fresh session should be able to resume from it alone.
4. Never fake a feature: no dead buttons and no mock data in shipped code. If a platform can't do something, ship the best available behaviour, show the limitation in the UI and document it.
5. Before adding a dependency, check that it is maintained, works on every target platform and has an acceptable licence. Flag anything GPL or commercial to me.
6. Ask me only about decisions that are expensive to reverse and not settled here. Otherwise decide, record the decision in ARCHITECTURE.md and continue.

Architecture and stack

Use one shared Rust core under every app, with a web-technology UI so theming and layout stay fully data-driven. This is my recommended stack; propose changes in ARCHITECTURE.md if you have strong reasons.

- **Monorepo:** `` (Rust), `ui/` (shared front end), `apps/` (desktop and mobile shells) and `docs/`.
- **Core (Rust):** audio engine, decoders, tag reading and writing, library database, scanner, queue and playback state, and the sync engine. Playback logic lives here, not in the UI, so it keeps running while the UI is suspended.
- **Apps:** Tauri 2 shells for Windows, macOS, Linux, iOS and Android, sharing one TypeScript + React UI.
- **Design tokens:** every colour, radius, spacing value, font, blur and shadow comes from CSS custom properties, so themes and layouts change without code.
- **Native plugins** wherever a webview can't do the job: audio output backends, background audio and sync, lock-screen and system media controls, and USB DAC handling.
- **Mobile fallback:** if Tauri can't meet the background-audio and system-integration requirements, use native UIs (SwiftUI and Jetpack Compose) over the same core via UniFFI. Settle this with a spike during planning.
- **Sync engine:** part of the core on every device. It talks to cloud storage APIs directly, not through the provider's own sync app, which can clash with tag writes.
- **Storage:** SQLite (WAL mode, FTS5 search) on every device, each holding a full replica of the library catalogue.
- **Libraries to evaluate** (verify each is current and maintained):

  - Symphonia for decoding, with FFmpeg (LGPL, dynamically linked) for formats it lacks.
  - lofty for tags, filling any format gaps it has.
  - rubato or libsoxr for resampling, ebur128 for loudness and Chromaprint for fingerprints.
  - Per-OS output crates (wasapi, coreaudio-rs, alsa) rather than a lowest-common-denominator audio layer.

Audio engine: hi-res playback

Playback must be bit-perfect when I want it, gapless always, and transparent about exactly what happens to the signal.

- **Formats:** FLAC, ALAC, WAV, AIFF, WavPack, Monkey's Audio (APE), DSD (DSF and DFF), MP3, AAC, Opus and Ogg Vorbis. Also single-file albums with CUE sheets, and multichannel files (downmixed when the device is stereo).
- **Resolutions:** 16-, 24- and 32-bit integer and 32-bit float PCM from 44.1 kHz to 384 kHz (768 kHz where the device supports it), and DSD64 to DSD256 (DSD512 where supported).
- **Bit-perfect mode:** exclusive device access with no processing. Use WASAPI exclusive mode on Windows, CoreAudio sample-rate switching (optional hog mode) on macOS, and ALSA `hw` devices on Linux. On mobile, use Android 14+ bit-perfect mixer attributes for USB DACs and the preferred sample rate on iOS.
- **Automatic sample-rate switching:** the device follows each track's native rate, including across gapless boundaries where the hardware allows.
- **Shared mode** for everyday use, with a high-quality resampler when the device rate differs.
- **DSD:** DoP to DACs that support it; otherwise high-quality DSD-to-PCM conversion.
- **Processing path:** 64-bit float whenever any DSP is active, and TPDF dither whenever bit depth is reduced.
- **Gapless:** sample-accurate, honouring encoder delay and padding (LAME headers, iTunSMPB) in MP3 and AAC.
- **Signal path panel:** every stage from file to device (format, bit depth, sample rate, decoder, each DSP step, resampling, output mode, device), with a clear verdict: bit-perfect, lossless (processed) or lossy.
- **Quality badges:** Lossless, Hi-Res and DSD badges in lists and Now Playing, with the exact spec on hover (e.g., FLAC · 24-bit/96 kHz · 2,950 kbps).
- **Sound tools** (each one turns off bit-perfect and says so in the signal path):

    - ReplayGain and EBU R128 normalisation by track, album or automatic (album when an album plays in order, track on shuffle), with clipping prevention.
    - 10-band graphic and parametric EQ with presets, AutoEq headphone profile import and per-device profiles.
    - Convolution with FIR impulse responses, such as room-correction filters.
    - Headphone crossfeed, crossfade (automatically off for gapless albums) and a volume limiter.
- **Output devices:** a picker in the player bar, with per-device settings for exclusive mode, maximum sample rate, resampling policy (never, when needed, or always to a chosen rate), DSD mode and volume type (hardware, software, or fixed at 100%).
- **Real-time safety:** the audio thread never allocates, locks or touches the disk; decoding runs ahead into a lock-free buffer. Local seeks finish in under 100 ms.

Library, tags and metadata

Metadata is the heart of the app: read and write every common tag format without loss, and model the library the way serious collectors organise music.

- **Scanning:** watch folders with incremental background scans (file-system events plus periodic verification). The UI stays usable while the first scan runs.
- **Moves and renames** are detected without losing play counts or playlist membership (match by path, then by audio content hash).
- **Tag formats:** ID3v2.3 and v2.4 (MP3, AIFF, WAV, DSF), Vorbis comments (FLAC, Ogg, Opus), MP4 atoms (M4A, ALAC, AAC) and APEv2 (WavPack, APE). The ID3 version written is a setting (v2.4 by default, v2.3 for older devices).
- **Never lose data:** preserve fields the app doesn't understand, and never drop unknown frames on write.
- **Fields:** title, artists, album, album artist, composer, lyricist, conductor, performers, genres, date and original date, track and disc number and total, disc subtitle, compilation, grouping, work and movement, BPM, key, mood, comment, plain and time-synced lyrics, ISRC, barcode, catalogue number, label, release type and country, MusicBrainz IDs, ReplayGain values, play count, every sort field, and arbitrary custom fields.
- **Multi-value fields** (several artists or genres) are stored and edited as lists, and written in each format's own convention.
- **Legacy encodings:** read old non-UTF-8 tags (Latin-1, Shift-JIS, CP1251) correctly, with a tool to fix mis-encoded text.
- **Artwork:** embedded and folder images, multiple picture types, full-resolution viewing, and cached thumbnails at several sizes.

- **Library model:** artists, albums, songs, genres, composers and playlists (with folders). Albums group by album artist, title and release identity, so same-named albums never merge. Editions of one album (remaster, deluxe) can sit together under "Other versions".
- **Classical music:** work and movement display, composer browsing, and performer, conductor and orchestra credits.
- **Song Info window** with tabs for Details, Artwork, Lyrics, Options (start and stop time, skip when shuffling, volume adjustment), Sorting, and File (codec, bit depth, sample rate, bitrate, channels, size, path, checksum, ReplayGain, date added, play history).
- **Favourites:** a favourite flag, optionally written to the file tags.
- **Search:** instant results across all fields as I type, an advanced syntax (`artist:"Radiohead" year:>2000 format:flac`) and saved searches.

Batch tag editor

The batch editor is a full workspace, not a dialog: it should match the power of Mp3tag and MusicBrainz Picard inside a modern UI.

- **Open it from any selection:** songs, albums, artists, a playlist or a search result.
- **Two editing modes:** a form (shared values shown, differing values shown as ‹Mixed›) and a spreadsheet-style grid with inline editing, multi-cell paste and fill-down.
- **Only changed fields are written;** every untouched field keeps each file's own value.
- **Actions:**
  - Find and replace (plain or regex), scoped to chosen fields.
  - Case conversion with an exceptions list; trim and clean whitespace and characters.
  - Tags from filename and filename from tags, using patterns such as `%albumartist%/%year% - %album%/%disc%-%track% %title%`.
  - Auto-number tracks and discs; split, merge and copy fields; copy tags from one track to others.
  - Clear fields, remove all tags and fix legacy encodings.
- **Artwork in bulk:** add, replace, remove, extract to the folder, and resize or recompress embedded art to a target size.
- **Online lookup:** match albums against MusicBrainz (artwork from the Cover Art Archive), with Discogs as an option, and identify untagged tracks by AcoustID fingerprint. Show a side-by-side diff and let me accept per field, per track or all at once.
- **Action presets:** save chains of the operations above and run them on any selection.

- **Preview before writing:** before and after for every file and field. Writes then run in the background with progress, cancel and an error report.
- **Undo:** full undo and redo, a change history, and one-click revert of any past batch.
- **Safe writes:** write to a temporary file, verify it, then atomically replace the original. Never re-encode or touch audio data, offer optional backups, and handle read-only and locked files gracefully.
- **File organiser:** rename and move files into folders from tags with the same pattern language, with preview and illegal-character and long-path handling. The library updates without losing any stats.

Design: inspired by Apple Music

Aim for the feel of Apple Music (calm, artwork-forward and typographically confident) with original branding. Use no Apple names, logos, icons or assets, and don't make a pixel-for-pixel copy.

- **Desktop layout:** a translucent sidebar (search at the top; Home; library sections for Recently Added, Artists, Albums, Songs, Genres and Composers; then playlists), the main content area, and a persistent player bar. The bar holds artwork, title and artist, transport, scrubber, volume, the output picker, and lyrics and queue toggles.
- **Home:** horizontal shelves of artwork cards.
- **Album grid:** large square artwork with softly rounded corners and a subtle shadow, title and artist beneath, a play button on hover, and a size slider.
- **Album page:** big artwork; title; artist in the accent colour, linking to the artist; genre · year · quality badge; prominent Play and Shuffle pill buttons; a clean tracklist with numbers, featured artists and durations; a footer with release date, track count, total length and label.
- **Artist page:** a hero image, top songs by my plays, then albums, EPs and singles, live albums, compilations and "appears on".
- **Now Playing (full screen):** a slowly moving gradient built from the artwork's colours, large artwork that eases smaller when paused, and big transport controls and scrubber.
- **Lyrics view:** time-synced, large type, the current line highlighted, smooth scrolling, and tap any line to seek.
- **Materials and type:** translucent glass surfaces with background blur, and the system font stack (San Francisco via `system-ui` on Apple platforms, Inter elsewhere; never bundle Apple fonts). Bold large titles, clear hierarchy and generous spacing.
- **Colour:** neutral surfaces with one accent (default a warm pink-red), changeable or derived from the current artwork.

- **Icons:** one open-source set (e.g., Lucide or Phosphor) with a consistent stroke weight.
- **Motion:** quick, subtle spring animations that honour Reduce Motion and Reduce Transparency.
- **Appearance:** light, dark and automatic, all equally polished.
- **Interaction:** context menus everywhere (right-click or long-press: Play Next, Play Later, Add to Playlist, Go to Album or Artist, Get Info, Edit Tags, Show in Folder, Favourite, Rate). Drag and drop works for queueing, playlists, importing files and dropping artwork onto albums.
- **Compact modes:** a mini player (with always-on-top) and a menu-bar or tray controller.
- **Mobile:** a tab bar (Home, Library, Search, plus customisable tabs), a floating mini player above it, and a swipe-up Now Playing with the same artwork, lyrics and queue views.

Customisation: layout and look

Every visual choice should be adjustable without code, and power users should be able to go further with themes and CSS.

Look

- **Appearance controls:** light, dark or automatic; accent colour (presets, custom, or from the current artwork); background style (solid, translucent glass, artwork-tinted); blur and transparency strength; corner radius; density (compact, comfortable, spacious); font family and size; artwork size.
- **Theme editor:** every design token editable with a live preview. Save, duplicate, import and export themes as JSON files, with a warning when text contrast falls below WCAG AA.
- **Built-in themes:** six to eight, e.g., Default Light, Default Dark, OLED Black, High Contrast, Paper and a few colourful ones.
- **Custom CSS** on desktop, applied after the theme, with a safe-mode start that ignores it.

Layout

- **Sidebar:** resizable and collapsible; reorder, hide and pin items and playlists.
- **Player bar:** top or bottom; compact, standard or large; choose which controls appear.
- **Dockable panels** (queue, lyrics, info and credits, tag editor, signal path): left, right, bottom or floating, all resizable.
- **Views:** a default view per section (grid, list or column browser). List columns can be added, removed, reordered, resized and sorted, with grouping options.

- **Now Playing styles:** artwork, lyrics-first, minimal or visualizer.
- **Layout presets:** named layouts (e.g., Listening, Tagging, Minimal), switchable from a menu or a shortcut.
- **Mobile:** reorder tabs, choose the Home shelves and pick the mini-player style.
- **Shortcuts:** fully remappable keyboard shortcuts and global media hotkeys.
- **Sync:** themes and layouts optionally sync across devices, with per-device overrides.

Cross-device library and sync

Every device holds a full copy of the library, and a change made on any device reaches all the others through my cloud storage.

- **Every device is a full replica:** its own audio files, artwork and database. No device is the primary, and each one keeps working offline.
- **Cloud storage is the hub:** the app connects directly to my cloud account (Dropbox, Google Drive, OneDrive, S3-compatible storage such as Backblaze B2, or WebDAV for Nextcloud and NAS boxes). It holds the audio, artwork and change log, so devices never need to be online at the same time, and it doubles as an off-site backup.
- **What syncs:** added, deleted, moved and renamed files; every tag and artwork edit; playlists and folders; favourites, play counts and history; lyrics; and optionally themes, layouts and settings.
- **Change log:** each device writes its own log segments (most cloud storage has no atomic append), with periodic snapshots for fast catch-up. Use each provider's change feed where one exists, and poll otherwise.
- **Tag edits sync as metadata, not files:** identify each track by a hash of its audio stream (tags excluded), send each edit as a small operation, and let every device rewrite its own copy. A one-field fix on a 200 MB file then moves a few bytes, not the file.
- **New files** upload once (resumable, deduplicated by audio hash, and within each provider's rate limits), then download to every other device.
- **Paths:** each device can keep its library in a different folder; paths sync relative to the library root, so organiser moves apply everywhere.
- **Safe deletes:** a deletion moves the file to an in-app trash on every device for 30 days before anything is permanently removed.
- **Conflicts:** plays are stored as events, so counts add up; favourites and tag fields use per-field last-writer-wins with hybrid logical clocks; playlist edits are mergeable operations. Anything ambiguous goes to a Sync Conflicts view for me to resolve.
- **Home-network fast path:** devices on the same network find each other (mDNS/Bonjour) and copy files directly instead of through the cloud.

- **Phones and small drives:** a full mirror by default, but each device can set a storage limit. Over the limit, it keeps the albums and playlists I choose, optionally as space-saving AAC or Opus copies that still get every tag edit; everything else stays browsable and streams on demand.
- **Adding a device:** sign in to the cloud account and pick a library folder. The catalogue is browsable at once, and audio downloads in the background, starting with what I play.
- **Sync controls:** per-device status and progress, pause and resume, bandwidth limits, and Wi-Fi-only and charging-only options on mobile.
- **Privacy:** optional client-side encryption, so the cloud provider only stores encrypted data.
- **Handoff and remote control:** move the current queue and position between devices through the sync log; on the same network, control any device from another (e.g., my phone as a remote for my desktop).

Mobile apps

The phone apps must behave like first-class music players, not wrapped web pages.

- Background playback with lock-screen, Control Centre and notification controls.
- Headset, Bluetooth and car controls, with correct audio-focus and interruption handling (calls, other apps).
- USB DAC support through the best bit-perfect path each OS allows, with a clear indicator when output is resampled.
- A full or partial library mirror (see sync), with background syncing as far as each OS allows.
- Stretch goals: CarPlay and Android Auto browsing, home-screen widgets and watch controls.

Extra features

These go beyond the core; they're what make a player feel complete.

Listening and discovery

- **Home from my own library:** Recently Played, Recently Added, Heavy Rotation, Favourites, Rediscover (favourites unplayed for six months), Released on This Day and random album picks.
- **Smart playlists:** nested rules on any field, play stats, dates and audio quality, with limits, sorting and live updating.

- **Queue:** Play Next and Play Later, drag to reorder, history, save as playlist, shuffle by song or album, repeat one or all, and Autoplay that continues with similar music from my library.
- **Library radio:** an endless mix from a seed song, album or artist, using genre, year, BPM, key and listening history.
- **Stats:** listening statistics and a yearly recap.
- **Lyrics:** embedded, sidecar `.lrc` files or fetched from LRCLIB, plus an editor with tap-to-sync timing.
- **Credits:** composers, producers and performers from tags and MusicBrainz.
- **Sleep timer:** after a set time, the current song or the current album.
- **Internet radio:** Icecast and SHOUTcast stream URLs.

Library tools

- **Library health:** duplicates (by tags or fingerprint), missing or low-resolution artwork, inconsistent album artists, missing tags, and broken or changed files.
- **Quality check:** a spectrogram viewer and detection of likely lossy-sourced or upsampled "hi-res" files.
- **Converter:** transcode selections to FLAC, ALAC, MP3, AAC or Opus with tags and artwork preserved.
- **Import and export:** Apple Music/iTunes library XML (play counts, date added, playlists); M3U, M3U8, PLS and XSPF playlists; library export to CSV or JSON; backup and restore.

Integrations

- **Scrobbling** to Last.fm and ListenBrainz, cached while offline.
- **Casting** to Chromecast and UPnP/DLNA renderers, and AirPlay through the system picker on Apple platforms.
- **System media controls** everywhere: media keys, SMTC on Windows, Now Playing on macOS and iOS, MPRIS on Linux and MediaSession on Android.
- **Visualizer:** an optional spectrum or oscilloscope mode for Now Playing.

Quality bar

Treat these as acceptance criteria for every milestone.

- **Performance:** 200,000 tracks run smoothly, with virtualised lists at 60 fps, search results in under 50 ms, cold start under 2 s, and an incremental rescan of an unchanged library in seconds.

- **Reliability:** never corrupt or lose files or user data; the database is crash-safe; nothing blocks the UI thread.
- **Accessibility:** complete keyboard navigation, screen-reader labels, visible focus and text scaling, with the OS accessibility settings honoured.
- **Privacy:** no telemetry and no account needed for local use; every online feature is opt-in; keys and tokens live in the OS keychain.
- **Test fixtures:** generate a test library by script (FFmpeg or SoX, no copyrighted music) covering every format, bit depth, sample rate, tag type and edge case (Unicode, multi-value fields, huge artwork, corrupt files).
- **Tests that must exist:**
  - Tag round-trips for every format, including preservation of unknown fields.
  - Bit-perfect: decoded PCM matches FLAC's embedded MD5, and captured output matches the source in bit-perfect mode.
  - Gapless: a continuous tone split across tracks plays with no discontinuity.
  - Interrupted tag writes never leave a damaged file.
  - Sync: devices that edit offline converge when they reconnect, interrupted transfers resume, and deleted files stay recoverable.
- **Packaging:** signed installers (MSIX or MSI, notarised DMG, AppImage or Flatpak plus .deb), auto-update, and iOS and Android builds.

Milestones

Build in this order; each milestone ends with a usable app.

1. **Foundation:** core crate, database, scanner, tag reading for all formats, the app shell (sidebar, album grid, songs list, search) and gapless shared-mode playback.
2. **Hi-res engine:** bit-perfect output on each desktop OS, sample-rate switching, DSD, the signal path panel, output settings, ReplayGain, EQ, convolution, crossfeed and crossfade.
3. **Design polish:** album and artist pages, Now Playing with animated backgrounds and synced lyrics, the mini player, light and dark appearance, and motion.
4. **Tag editing:** the Song Info window, batch editor, actions and presets, online lookup, artwork tools, undo and the file organiser.
5. **Customisation:** the theme engine and editor, layout system and presets, custom CSS And shortcuts. Accent colour picker. Reorderable grid/tab/list options. Gradients. Liquid glass option, refer to the internet for how to implement it on android and linux.

6. **Sync:** cloud storage connectors, the change log, metadata-level tag sync, resumable and deduplicated file transfer, conflict handling, the home-network fast path, storage limits, and handoff and remote control.

7. **Mobile:** iOS and Android apps with full or partial mirrors, background playback and sync, system controls and USB DAC support.
8. **Extras:** Home shelves, smart playlists, stats, library radio, the lyrics editor, scrobbling, casting, library health, quality check, converter and imports.
9. **Hardening:** a performance pass against the budgets, an accessibility audit, packaging, auto-update and user documentation.

Definition of done

A milestone is done when its features work end to end on every platform it targets, its tests pass, no placeholder UI remains, and PROGRESS.md and the docs are updated. Finish each milestone with a short demo script: what to click to see each new feature.